

CTF Cafe

Season V US Cyber Open Beginner's Game Room

Introduction

The challenge is a Reverse Engineering challenge titled "CTF Cafe". The description is as follows:

Welcome to the CTF Cafe, where we serve only the finest menu items! We pride ourselves in our secret sauce, which is *exclusive or* something along those lines. At least, I think it is...

The file provided is named **ctf_cafe**. Given this is a reverse engineering challenge, this is likely an executable file. Running the Linux file command on it, we receive the following output:

```
$ file ctf_cafe
ctf_cafe: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=1534b5bd3a48183fc05ce7a8fe6d5f02d82581fb, for GNU/Linux
3.2.0, not stripped
```

This gives us some crucial information: it's a Linux executable, for the x86-64 architecture (meaning most modern desktop/laptop computers), and it's not stripped, meaning function names will be intact if we open it with a reverse engineering tool such as Ghidra.

Let's start by getting ourselves familiar with the application itself by running it on a Linux machine before throwing it into a reverse engineering tool.

```
$ ./ctf_cafe
```

```
===== 🍔 Welcome to the CTF Cafe! =====
```

1. View Menu
2. Place Order
3. View Total
0. Exit

Enter your choice:

From first glance, it looks like we have a way to view a menu, place an order, and view the total cost. Let's check out the menu:

Enter your choice: 1

--- Our Menu ---

- 1. Burger - \$5.99
- 2. Pizza - \$8.99
- 3. Salad - \$4.99
- 4. Soda - \$1.99
- 5. Ice Cream - \$2.99

You can place some orders, view your total, but nothing really seems to change. This application looks normal, right?

Examining the Strings

By running the strings tool on the application, we see some unusual stuff:

```
$ strings ./ctf_cafe
```

```
...
```

```
Oh, so you want the secret sauce recipe? Only if you have our  
proprietary key!
```

```
Enter 8-byte hex key (e.g., 0x0123456789ABCDEF):
```

```
Invalid input.
```

```
Incorrect key! You can't have the secret sauce recipe.
```

```
Congratulations! You have unlocked the secret sauce recipe!
```

```
Secret Sauce:
```

These strings look like they're referencing "secret sauce", mentioned in the description of the challenge, and a hex key. These never appear anywhere in the visible menu, so it's safe to assume there's something missing from the menus we are given.

First Look in Ghidra

After opening the application in Ghidra, we reach the **main()** function. When opening the decompilation for **main()**, we see the following:

```

1
2 undefined8 main(void)
3
4 {
5     int iVar1;
6     int local_10;
7     uint local_c;
8
9 LAB_004004ae:
10    while( true ) {
11        puts(&DAT_00401300);
12        puts("1. View Menu");
13        puts("2. Place Order");
14        puts("3. View Total");
15        puts("0. Exit");
16        printf("Enter your choice: ");
17        iVar1 = __isoc23_scanf(&DAT_00401371,&local_10);
18        if (iVar1 == 1) break;
19        clear_input();
20        puts("Invalid input. Please enter a number.");
21    }
22    if (local_10 == 9) {
23        puts("Oh, so you want the secret sauce recipe? Only if you have our proprietary key!");
24        printf("Enter 8-byte hex key (e.g., 0x0123456789ABCDEF): ");
25        iVar1 = __isoc23_scanf(&DAT_00401442,&key);
26        if (iVar1 != 1) {
27            puts("Invalid input.");
28            return 1;
29        }
30        if (key == -0x642d38a5b63b1015) {
31            puts("Congratulations! You have unlocked the secret sauce recipe!");
32            printf("Secret Sauce: ");
33            for (local_c = 0; local_c < 0x21; local_c = local_c + 1) {
34                putchar((int)(char) (*(byte *) ((long)&size + (long)((int)local_c % 8)) ^
35                                secret_sauce[(int)local_c]));
36            }
37            putchar(10);
38        }
39        else {
40            puts("Incorrect key! You can't have the secret sauce recipe.");
41        }

```

A pointer to **local_10** is passed to `scanf()`, the C function for reading and parsing values from strings. The first argument is a pointer to the data section, where strings are stored, meaning it is likely the format string associated with `scanf`. It isn't defined properly, but we can assume this is reading our choice as a number, since it comes directly after the "Enter your choice" print call and `local_10` is an integer variable. This means **local_10** is likely where the option we select is stored.

Afterwards, **local_10** is compared to the number 9, in which case it prints the "secret sauce" string. The menu didn't list 9 as an option, but let's try it:

```
$ ./ctf_cafe
```

```
===== 🍷 Welcome to the CTF Cafe! =====
```

1. View Menu
2. Place Order
3. View Total
0. Exit

```
Enter your choice: 9
```

```
Oh, so you want the secret sauce recipe? Only if you have our  
proprietary key!
```

```
Enter 8-byte hex key (e.g., 0x0123456789ABCDEF):
```

We are now being asked for an 8-byte hex key. Let's take a closer look at this branch of the code:

```
if (local_10 == 9) {  
    puts("Oh, so you want the secret sauce recipe? Only if you have our proprietary key!");  
    printf("Enter 8-byte hex key (e.g., 0x0123456789ABCDEF): ");  
    iVar1 = __isoc23_scanf(&DAT_00401442, &key);  
    if (iVar1 != 1) {  
        puts("Invalid input.");  
        return 1;  
    }  
    if (key == -0x642d38a5b63b1015) {  
        puts("Congratulations! You have unlocked the secret sauce recipe!");  
        printf("Secret Sauce: ");  
        for (local_c = 0; local_c < 0x21; local_c = local_c + 1) {  
            putchar((int)(char) (*(byte *) ((long) &size + (long) ((int) local_c % 8)) ^  
                        secret_sauce[(int) local_c]));  
        }  
        putchar(10);  
    }  
    else {  
        puts("Incorrect key! You can't have the secret sauce recipe.");  
    }  
}
```

We see the program reads something with the scanf function into the **key** variable. Again, the format string is not defined, but we can assume it is probably a hex-encoded number based on the surrounding prompts. The program then compares the value of the **key** variable with `-0x642d38a5b63b1015`. If the two values are equal, then it decrypts the “secret sauce”. Note that it is using the XOR operation (^), which is why “*exclusive or*” was emphasized in the challenge description.

Let's try inputting option 9 (the hidden option we found earlier), and then that hex key:

```
$ ./ctf_cafe
```

```
===== 🍔 Welcome to the CTF Cafe! =====
```

- 1. View Menu
- 2. Place Order
- 3. View Total
- 0. Exit

```
Enter your choice: 9
```

```
Oh, so you want the secret sauce recipe? Only if you have our  
proprietary key!
```

```
Enter 8-byte hex key (e.g., 0x0123456789ABCDEF): -0x642d38a5b63b1015
```

```
Congratulations! You have unlocked the secret sauce recipe!
```

```
Secret Sauce: SVBGR{d3c0mp1l3rs_m4k3_l1f3_34sy}
```

```
Invalid choice. Try again.
```

And we have our flag: SVBGR{d3c0mp1l3rs_m4k3_l1f3_34sy}. Indeed, they do!

Summary

To solve this challenge, you first had to identify a secret menu option by decompiling a Linux x86-64 executable, then further identify a key required by that secret option within the decompilation. After inputting the secret menu option along with the key, you would receive the flag.